

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



RASYA DIKA PRATAMA

109082500006

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pengurutan data (sorting) merupakan proses penting dalam pemrograman yang bertujuan untuk menyusun sekumpulan data acak menjadi berurutan dengan pola tertentu, baik itu membesar (ascending) dari indeks kecil ke besar, maupun mengecil (descending). Algoritma sorting sangat diperlukan untuk mempermudah dan mempercepat proses pencarian atau analisis data, terutama jika ukuran datanya sangat besar. Pada dasarnya, terdapat berbagai macam pendekatan untuk melakukan sorting, dan dua di antaranya yang paling fundamental adalah Selection Sort dan Insertion Sort.

Algoritma Selection Sort merupakan metode pengurutan seleksi yang ide utamanya adalah mencari nilai ekstrem (terkecil atau terbesar) pada sekumpulan data yang tersisa. Setelah nilai ekstrem tersebut ditemukan, algoritma akan memindahkan atau menukar posisinya dengan data yang berada pada urutan paling kiri pada rentang data tersebut (proses *swap*). Proses pencarian dan pertukaran ini diulangi terus-menerus hingga rentang data tersisa hanya satu elemen saja, yang menandakan bahwa seluruh kumpulan data telah berhasil diurutkan dengan benar.

Berbeda dengan Selection Sort, algoritma Insertion Sort tidak melakukan pencarian nilai ekstrem terlebih dahulu, melainkan bekerja dengan cara menyisipkan suatu nilai pada posisi yang seharusnya. Algoritma ini membagi data menjadi dua bagian imajiner: bagian yang sudah terurut dan yang belum. Algoritma akan mengambil satu data dari bagian yang belum terurut, menggeser data-data yang sudah terurut ke sebelah kanan untuk menciptakan satu ruang kosong, lalu menyisipkan data baru tersebut ke posisi yang tepat menggunakan pencarian sekuensial (sequential search). Proses penyisipan ini diulang hingga seluruh data yang belum terurut habis disisipkan.

B. Guided

1. Program Unguided 1

```
package main

import "fmt"

func selecetionSortArray(angka *[5]int) {
```

```

var idx_min, i, j int

for i = 0; i < len(angka)-1; i++ {
    idx_min = i
    for j = i + 1; j < len(angka); j++ {
        if angka[j] <= angka[idx_min] {
            idx_min = j
        }
    }
    if idx_min != i {
        angka[i], angka[idx_min] = angka[idx_min], angka[i]
    }
}

}

func main() {
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")

    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
}

```

```

    selecetionSortArray(&arrAngka)

    for i := 0; i < len(arrAngka); i++ {

        fmt.Print(arrAngka[i], " - ")

    }

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Guided\Guided-1
$ go run .\guided1.go
Masukkan data angka ke-0 : 9
Masukkan data angka ke-1 : 3
Masukkan data angka ke-2 : 7
Masukkan data angka ke-3 : 1
Masukkan data angka ke-4 : 5

=== SEBELUM DISORTING ===
9 - 3 - 7 - 1 - 5 -
=== SETELAH DISORITNG ===
1 - 3 - 5 - 7 - 9 -

```

Penjelasan:

Program ini mengimplementasikan algoritma Selection Sort untuk mengurutkan array berisi 5 angka secara ascending (membesar). Proses pengurutan menggunakan dua perulangan; perulangan pertama digunakan sebagai batas area elemen, sedangkan perulangan kedua bertugas untuk membandingkan nilai dan mencari indeks dengan nilai elemen terkecil. Jika indeks dengan nilai terkecil yang ditemukan berbeda dengan indeks awal iterasi saat itu, maka program akan melakukan proses penukaran posisi (*swap*) menggunakan *multiple assignment* bawaan Go, sehingga elemen terkecil berpindah ke bagian depan.

2. Program guided 2

```

package main

import "fmt"

```

```

type mahasiswa struct {
    nama, nim string
    IPK      float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d\n", i)

        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM  : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK  : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()
    fmt.Println(" === SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM  : %s\n", structMHS[i].nim)
        fmt.Printf("IPK  : %.2f\n", structMHS[i].IPK)
    }
}

```

```
fmt.Println("=====")
fmt.Println()
SelectionSortArray(&structMHS)
fmt.Println(" === SETELAH SORITNG === ")
for i := 0; i < len(structMHS); i++ {
    fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
    fmt.Printf("Nama : %s\n", structMHS[i].nama)
    fmt.Printf("NIM : %s\n", structMHS[i].nim)
    fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
}
fmt.Println("=====")
fmt.Println()
}
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  DEVDB  GITLens

$ go run .\guided2.go
Nama : nana
NIM : 1001
IPK : 3.20
--- Data Mahasiswa Ke-1 ---
Nama : budi
NIM : 1002
IPK : 3.85
--- Data Mahasiswa Ke-2 ---
Nama : cici
NIM : 1003
IPK : 2.95
--- Data Mahasiswa Ke-3 ---
Nama : cery
NIM : 1004
IPK : 3.60
--- Data Mahasiswa Ke-4 ---
Nama : kiasa
NIM : 1005
IPK : 3.30
=====

=== SETELAH SORITNG ===
--- Data Mahasiswa Ke-0 ---
Nama : cici
NIM : 1003
IPK : 2.95
--- Data Mahasiswa Ke-1 ---
Nama : nana
NIM : 1001
IPK : 3.20
--- Data Mahasiswa Ke-2 ---
Nama : kiasa
NIM : 1005
IPK : 3.30
--- Data Mahasiswa Ke-3 ---
Nama : cery
NIM : 1004
IPK : 3.60
--- Data Mahasiswa Ke-4 ---
Nama : budi
NIM : 1002
IPK : 3.85
=====

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Guided\Guided-2
$ cd ..\Guided-3\
```

Penjelasan:

Program ini menggunakan metode Selection Sort untuk mengurutkan array tipe data bentukan (struct mahasiswa) berdasarkan atribut IPK secara ascending. Data yang diinputkan awalnya tidak terurut, kemudian algoritma menyeleksinya dengan mencari nilai IPK terkecil pada blok perulangan terdalam. Setelah IPK terkecil ditemukan, seluruh entitas record mahasiswa pada indeks tersebut (termasuk nama dan NIM) akan saling bertukar posisi

(*swap*) ke urutan depan array, sehingga pada akhir eksekusi, seluruh data mahasiswa akan tercetak berurutan dari IPK terendah hingga tertinggi.

3. Program guided 3

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
    insertionSort(&data, n)
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}
```


Output :

```
dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Guided\Guided-3
$ go run .\guided3.go
7
12
3
45
8
21
6
30
45 30 21 12 8 6 3
```

Penjelasan:

Program ini mendemonstrasikan algoritma Insertion Sort untuk mengurutkan sekumpulan elemen integer di dalam array secara descending (mengecil). Program berjalan dengan cara menyimpan elemen indeks saat ini ke dalam variabel sementara (temp), kemudian secara sekuensial mengecek dan membandingkannya dengan elemen-elemen sebelumnya. Selama kondisi perulangan terpenuhi, yaitu elemen sebelumnya lebih kecil dari nilai temp maka elemen tersebut akan terus digeser ke kanan, baru kemudian program menyisipkan nilai temp ke ruang kosong yang tepat agar urutan besarnya tetap terjaga.

4. Program guided 4

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int
    i = 1
    for i <= n-1 {
```

```

        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int
    fmt.Scan(&n)
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
    insertionSortString(&data, n)
    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Guided\Guided-4
$ go run .\guided4.go
5
cherry
durian
kelapa
jeruk
pisang
cherry durian jeruk kelapa pisang

```

Penjelasan:

Serupa dengan program sebelumnya, program ini mengurutkan tipe data string yang berisi nama-nama buah, namun kali ini diurutkan secara ascending berdasarkan urutan leksikografis (abjad). Algoritma Insertion Sort

beroperasi dengan menyeleksi satu per satu kata, menaruhnya di variabel temp, dan membandingkannya dengan kata-kata di posisi sebelumnya. Apabila string yang berada di sebelah kiri secara abjad bernilai lebih besar (lebih akhir), maka posisi string tersebut akan digeser ke kanan, untuk memberikan tempat bagi temp agar bisa disisipkan ke posisi alfabetis yang benar di sebelah depan.

C. Unguided

1. Soal Unguided 1

Jawaban :

```
package main

import "fmt"

const NMax = 1000000
type arrInt [NMax]int

func selectionSortAsc(T *arrInt, n int) {
    var t, i, j, idx_min int
    i = 1
    for i <= n-1 {
        idx_min = i - 1
        j = i
        for j < n {
            if T[idx_min] > T[j] {
                idx_min = j
            }
        }
    }
}
```

```

        j = j + 1

    }

    t = T[idx_min]

    T[idx_min] = T[i-1]

    T[i-1] = t

    i = i + 1

}

}

func main() {

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        var m int

        fmt.Scan(&m)

        var arr arrInt

        for j := 0; j < m; j++ {

            fmt.Scan(&arr[j])

        }

        selectionSortAsc(&arr, m)

        for j := 0; j < m; j++ {

            fmt.Print(arr[j], " ")

        }

        fmt.Println()
    }
}

```

```
}  
}
```

Output :

```
dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Unguided\Unguided-1  
$ go run .\soal1.go  
3  
5 2 1 7 9 13  
1 2 7 9 13  
6 189 15 27 39 75 133  
15 27 39 75 133 189  
3 4 9 1  
1 4 9
```

Penjelasan :

Program ini menerapkan fungsi algoritma Selection Sort (ascending) untuk menyusun data nomor rumah kerabat dari sekumpulan daerah yang diinputkan dinamis. Program akan menerima masukan jumlah daerah (n) lalu di tiap iterasinya akan membaca jumlah rumah (m) beserta kumpulan nomor rumah yang lalu ditampung ke dalam array integer. Setelah data masuk, array dipassing menggunakan sistem pointer ke dalam fungsi selectionSortAsc untuk diurutkan dengan prinsip mencari dan menukar nilai terkecil ke indeks sisi kiri secara kontinu, sebelum akhirnya keseluruhan nomor rumah yang terurut dicetak ke layar per daerah.

2. Soal Unguided 2

Jawaban :

```
package main

import "fmt"

const NMax = 1000000

type arrInt [NMax]int

func selectionSortAsc(T *arrInt, n int) {

    var t, i, j, idx_min int

    i = 1

    for i <= n-1 {

        idx_min = i - 1

        j = i

        for j < n {

            if T[idx_min] > T[j] {

                idx_min = j

            }

            j = j + 1

        }

        t = T[idx_min]

        T[idx_min] = T[i-1]

        T[i-1] = t

    }

}
```

```
        i = i + 1

    }

}

func selectionSortDesc(T *arrInt, n int) {

    var t, i, j, idx_max int

    i = 1

    for i <= n-1 {

        idx_max = i - 1

        j = i

        for j < n {

            if T[idx_max] < T[j] {

                idx_max = j

            }

            j = j + 1

        }

        t = T[idx_max]

        T[idx_max] = T[i-1]

        T[i-1] = t

        i = i + 1

    }

}

func main() {

    var n int

    fmt.Scan(&n)
```

```
for i := 0; i < n; i++ {  
  
    var m int  
  
    fmt.Scan(&m)  
  
    var arrGanjil, arrGenap arrInt  
  
    var nGanjil, nGenap int  
  
    for j := 0; j < m; j++ {  
  
        var val int  
  
        fmt.Scan(&val)  
  
        if val%2 != 0 {  
  
            arrGanjil[nGanjil] = val  
  
            nGanjil++  
  
        } else {  
  
            arrGenap[nGenap] = val  
  
            nGenap++  
  
        }  
    }  
}  
  
selectionSortAsc(&arrGanjil, nGanjil)  
selectionSortDesc(&arrGenap, nGenap)  
  
for j := 0; j < nGanjil; j++ {  
  
    fmt.Print(arrGanjil[j], " ")  
}  
  
for j := 0; j < nGenap; j++ {  
  
    fmt.Print(arrGenap[j], " ")
```



```

    }

    fmt.Println()

}
}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Unguided\Unguided-2
$ go run .\soal2.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program ini memiliki fungsionalitas memisahkan nomor rumah kerabat berdasar sifat bilangan ganjil dan genap ke dalam dua array penampung berbeda langsung pada saat proses pembacaan perulangan *input* dilakukan. Array khusus data nomor rumah ganjil akan diurutkan membesar dengan fungsi `selectionSortAsc`, sementara array penampung nomor rumah genap akan diurutkan mengecil memanfaatkan fungsi `selectionSortDesc`. Setelah kedua operasi modifikasi elemen array tersebut selesai, hasil cetaknya akan digabung dalam satu baris, mencetak deretan nomor ganjil dari terkecil ke terbesar lebih dulu dan langsung diikuti nomor genap dari terbesar ke terkecil.

3. Soal Unguided 3

Jawaban :

```
package main

import "fmt"

const NMax = 1000000

type arrInt [NMax]int

func insertionSortAsc(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}
```

```
func main() {  
  
    var arr arrInt  
  
    var n, val int  
  
    for {  
        fmt.Scan(&val)  
  
        if val < 0 {  
            break  
        }  
  
        arr[n] = val  
  
        n++  
    }  
  
    insertionSortAsc(&arr, n)  
  
    for i := 0; i < n; i++ {  
        fmt.Print(arr[i], " ")  
    }  
  
    fmt.Println()  
  
    if n > 1 {  
        jarak := arr[1] - arr[0]  
  
        tetap := true  
  
        for i := 1; i < n-1; i++ {  
            if arr[i+1]-arr[i] != jarak {  
                tetap = false  
            }  
        }  
    }  
}
```

```

        break
    }

}

if tetap {

    fmt.Printf("Data berjarak %d\n", jarak)

} else {

    fmt.Println("Data berjarak tidak tetap")

}

}

}

```

Output :

```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Unguided\Unguided-3
$ go run .\soal3.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Unguided\Unguided-3
$ go run .\soal3.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program ini dibuat agar secara terus-menerus membaca dan mengumpulkan masukan bilangan bulat dari *user* ke dalam array, di mana proses perulangan *input* baru akan dihentikan secara paksa (*break*) apabila program mendeteksi angka bernilai negatif. Array yang sudah terbentuk ini diurutkan membesar (ascending) menggunakan fungsi pembantu `insertionSortAsc` dengan prinsip pergeseran elemen ke kanan. Usai pengurutan dan pencetakan, program melakukan iterasi validasi terakhir untuk menghitung selisih antarelemen berdekatan; jika iterasi menemukan semua jaraknya konsisten, program mencetak angka jarak tersebut, sebaliknya akan dicetak bahwa "Data berjarak tidak tetap".

4. Soal Unguided 4

Jawaban :

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit,
&pustaka[i].eksemplar, &pustaka[i].tahun, &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
```

```

        return

    }

    maxIdx := 0

    for i := 1; i < n; i++ {

        if pustaka[i].rating > pustaka[maxIdx].rating {

            maxIdx = i

        }

    }

    fmt.Println(pustaka[maxIdx].judul, pustaka[maxIdx].penulis,
pustaka[maxIdx].penerbit, pustaka[maxIdx].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {

    var temp Buku

    var i, j int

    i = 1

    for i <= n-1 {

        j = i

        temp = pustaka[j]

        for j > 0 && temp.rating > pustaka[j-1].rating {

            pustaka[j] = pustaka[j-1]

            j = j - 1

        }

        pustaka[j] = temp

        i = i + 1

    }

}

```

```
func Cetak5Terbaru(pustaka DaftarBuku, n int) {  
    batas := 5  
    if n < 5 {  
        batas = n  
    }  
    for i := 0; i < batas; i++ {  
        fmt.Println(pustaka[i].judul)  
    }  
}  
  
func CariBuku(pustaka DaftarBuku, n int, r int) {  
    kiri := 0  
    kanan := n - 1  
    ketemu := false  
    var tengah int  
  
    for kiri <= kanan && !ketemu {  
        tengah = (kiri + kanan) / 2  
        if pustaka[tengah].rating == r {  
            ketemu = true  
        } else if r > pustaka[tengah].rating {  
            kanan = tengah - 1  
        } else {  
            kiri = tengah + 1  
        }  
    }  
}
```

```

    }

    if ketemu {

        fmt.Println(pustaka[tengah].judul,
pustaka[tengah].penulis, pustaka[tengah].penerbit,
pustaka[tengah].tahun, pustaka[tengah].eksemplar,
pustaka[tengah].rating)

    } else {

        fmt.Println("Tidak ada buku dengan rating seperti itu")

    }

}

func main() {

    var pustaka DaftarBuku

    var n, ratingCari int

    DaftarkanBuku(&pustaka, &n)

    fmt.Scan(&ratingCari)

    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)

    Cetak5Terbaru(pustaka, n)

    CariBuku(pustaka, n, ratingCari)

}

```

Output :


```

dikadev at ACER-NITRO-5 in D:\Tel-u\semester-2\Week_12\Unguided\Unguided-4
$ go run .\soal4.go
6
B01 Laskar_Pelangi Andrea_Hirata Bentang_Pustaka 100 2005 95
B02 Bumi_Manusia Pramoedya_A_T Hasta_Mitra 50 1980 98
B03 Cantik_Itu_Luka Eka_Kurniawan Gramedia 75 2002 90
B04 Pulang_Tere_Liye Republika 120 2015 88
B05 Hujan_Tere_Liye Gramedia 150 2016 92
B06 1984 George_Orwell Secker 80 1949 96
92
Bumi_Manusia Pramoedya_A_T Hasta_Mitra 1980
Bumi_Manusia
1984
Laskar_Pelangi
Hujan
Cantik_Itu_Luka
Hujan_Tere_Liye Gramedia 2016 150 92

```

Penjelasan :

Program pengelola data buku perpustakaan ini sangat komprehensif karena tidak hanya berisi algoritma sorting, melainkan perpaduan dari fungsi pencarian indeks nilai tertinggi (CetakTerfavorit), algoritma Insertion Sort descending berdasarkan metrik rating (UrutBuku), manipulasi batasan cetak data array (Cetak5Terbaru), dan diakhiri dengan implementasi murni algoritma Binary Search (CariBuku). Data awal dibaca masuk ke sebuah *array of struct* tipe Buku, lalu dicari judul dengan rating paling mutlak tinggi, kemudian keseluruhan elemen array dirombak dan diurutkan dari rating tinggi ke rendah dengan sistem sisip, dicetak kembali hanya 5 elemen peraih posisi puncaknya, dan akhirnya ditutup dengan operasi pencarian belah-dua untuk menampilkan data rinci dari sebuah buku berdasarkan *rating* spesifik yang diinputkan pengguna di baris terbawah.

D. Kesimpulan

Kesimpulan dari praktikum Modul 14 Sorting ini adalah bahwa pengurutan data merupakan pilar algoritma fundamental dalam pemrograman untuk menyusun data secara terstruktur, yang manfaat akhirnya sangat signifikan untuk menyederhanakan proses pencarian maupun pembacaan data. Algoritma Selection Sort menawarkan alur logika pengurutan yang sangat lurus dan mudah dipahami, yakni berfokus pada pencarian nilai paling ekstrem secara berulang lalu melakukan operasi penukaran tempat (swap) ke batas posisi array yang sudah disiapkan. Sementara itu, Insertion Sort bekerja dengan cara yang lebih taktis secara sekuensial lewat pergeseran elemen dan penyisipan data pada

ruang kosong yang diciptakan, sehingga terbukti menjadi sangat stabil dan menjaga keterurutan dengan baik. Penguasaan atas implementasi modular menggunakan fungsi untuk kedua jenis sorting ini terlebih bila diaplikasikan pada kompleksitas Tipe Data Bentukan (Struct) dan digabung algoritma Binary Search menjadi kompetensi yang amat berharga bagi *developer* dalam merekayasa program sistem pendataan terpadu di dunia nyata.